

Experiment 1

```
<html>

<head>

<head>

  <title>XYZ Institute</title>

  <style>

    body { font-family: Arial, sans-serif; margin: 0; background: #f4f4f4; }

    header, footer, nav { background: #333; color: white; text-align: center; padding: 10px 0; }

    nav a { color: white; padding: 14px 20px; text-decoration: none; }

    nav a:hover { background: #575757; }

    section { padding: 20px; }

    .department { background: white; margin: 10px 0; padding: 15px; }

  </style>

</head>

<body>

  <header><h1>XYZ Institute</h1></header>

  <nav>

    <a href="#home">Home</a><a href="#departments">Departments</a><a href="#contact">Contact</a>

  </nav>

  <section id="home"><h2>Welcome</h2><p>We provide world-class education in various fields.</p></section>

  <section id="departments">

    <h2>Departments</h2>

    <div class="department"><h3>Computer Science</h3><p>Programs in software engineering, AI, etc.</p></div>

    <div class="department"><h3>Mechanical Engineering</h3><p>Focuses on machinery design.</p></div>

  </section>

</body>

</html>
```

```
<div class="department"><h3>Civil Engineering</h3><p>Specializes in construction and infrastructure.</p></div>
```

```
</section>
```

```
<section id="contact"><h2>Contact</h2><p>Email: contact@xyzinstitute.edu<br>Phone: +123 456 7890</p></section>
```

```
</body>
```

```
</html>
```

1. **HTML (HyperText Markup Language)** is the standard language for creating and structuring content on the web. It defines elements like text, images, links, and forms to build web pages. HTML forms the foundation for web development, allowing browsers to display content and enabling navigation and accessibility.

2. **<html>**: The root element that wraps all content in the document.

<head>: Contains metadata, like the document's title, links to stylesheets, and other resources.

<h1> to <h6>: Headings for structuring content, where **<h1>** is the highest level.

<title>: Sets the title of the webpage (appears in the browser tab).

<body>: Contains the visible content of the web page (e.g., text, images, links).

3. **CSS (Cascading Style Sheets)** is a language used to style and format the layout of web pages. It controls the appearance of HTML elements, such as colors, fonts, spacing, and positioning, making web pages visually appealing and responsive.
4. You can link an external stylesheet to your HTML document using the **<link>** tag inside the **<head>** section. Here's the syntax:

```
html
Copy code
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

5. The **<nav>** tag in HTML5 is used to define a section of navigation links. It helps organize the structure of the webpage, making it clear where the navigation is located. This improves SEO by allowing search engines to identify navigation areas, enhances accessibility for screen readers, and ensures better code organization and maintainability.

Experiment 2

```
<html>

<head>

  <title>Faculty Entry Form</title>

  <style>

    body { font-family: Arial, sans-serif; background-color: #f4f4f4; margin: 0; padding: 0;
  }

    h2 { text-align: center; }

    label { display: block; margin: 10px 0 5px; }

    button { background-color: #4CAF50; color: white; cursor: pointer; }

  </style>

</head>

<body>

  <div class="container">

    <h2>Faculty Details</h2>

    <form>

      <label>Name:</label>

      <input type="text" name="name" required><br>

      <label>Email:</label>

      <input type="email" name="email" required><br>

      <label>Phone:</label>

      <input type="tel" name="phone" required><br>

      <label>Department:</label>

      <select name="department" required>

        <option value="Computer Science">Computer Science</option>

        <option value="Mechanical Engineering">Mechanical Engineering</option>

        <option value="Civil Engineering">Civil Engineering</option>

      </select>

    </form>

  </div>

</body>

</html>
```

```

        </select><br>

        <label>Qualification:</label>

        <input type="text" name="qualification" required><br>

        <button type="submit">Submit</button>

    </form>

</div>

</body>

</html>

```

1. The **<form>** tag in HTML is used to collect user input and send it to a server for processing. It creates an interactive form on a web page where users can enter data, such as text, numbers, or selections.
2. In HTML, different **input elements** are used to capture various types of data from the user. They are differentiated by the **type** attribute, which defines the kind of input that is expected.
3. **action**: Specifies the URL where the form data will be sent for processing.
method: Defines how the data will be sent:

GET (appends data to the URL, less secure).

POST (sends data in the request body, more secure).

4. To add validation to an HTML form, use attributes like

required: Ensures a field is filled out.

type: Specifies the input format (e.g., `email`, `number`).

min / max: Sets a range for numbers or dates.

pattern: Defines a regex pattern for text input.

5. **GET** sends data in the URL, visible and limited in size, used for non-sensitive data like searches.
POST sends data in the request body, hidden and can handle large data, used for sensitive data like logins or form submissions.

Experiment 3

```
<html>

<head>

  <title>Tech Tutorials</title>

  <style>

    body { font-family: Arial, sans-serif; margin: 0; padding: 0; background: #f4f4f4; }

    header { background: #333; color: white; padding: 10px 0; text-align: center; }

    nav ul { text-align: center;}

    nav ul li a { color: black; text-decoration: none; }

    .section { padding: 20px; text-align: center; }

    article { background: white; padding: 15px; margin: 10px 0; }

    footer { background: #333; color: white; text-align: center; padding: 10px 0; }

    @media (max-width: 768px) {

      nav ul li { display: block; margin: 10px 0; }

      .section { padding: 15px; }

    }

  </style>

</head>

<body>

  <header><h1>Tech Tutorials Blog</h1></header>

  <nav>

    <ul>

      <li><a href="#home">Home</a></li>

      <li><a href="#tutorials">Tutorials</a></li>

      <li><a href="#about">About</a></li>

      <li><a href="#contact">Contact</a></li>

    </ul>
```

```
</nav>

<section id="home" class="section">

  <h2>Welcome to Tech Tutorials</h2>

  <p>Explore tutorials on web development and tech skills.</p>

</section>

<section id="tutorials" class="section">

  <h2>Latest Tutorials</h2>

  <article><h3>HTML & CSS Basics</h3><p>Learn the fundamentals of web
development with HTML and CSS.</p></article>

  <article><h3>JavaScript Basics</h3><p>Understand the basics of JavaScript
programming.</p></article>

</section>

<section id="about" class="section">

  <h2>About Us</h2>

  <p>We simplify learning tech for everyone.</p>

</section>

<section id="contact" class="section">

  <h2>Contact</h2>

  <p>Email: contact@techtutorials.com</p>

</section>

</body>

</html>
```

1. **Responsive web design** ensures that a website's layout and content adapt smoothly to different screen sizes and devices (e.g., desktops, tablets, smartphones). It uses flexible grids, images, and CSS media queries.

Importance: It provides a better user experience across all devices, improves accessibility, and boosts SEO rankings since search engines prioritize mobile-friendly sites.

2. **Media queries** in CSS allow you to apply styles based on the device's characteristics, like screen size, resolution, or orientation. They enable responsive design by tailoring the layout to different devices
3. **Flexbox** helps create responsive layouts by allowing elements to adjust their size, position, and alignment dynamically within a container.
4. Testing on multiple devices ensures consistent user experience, identifies layout issues, and improves accessibility across various screen sizes.
5. Common units for responsive design include %, **vw**, **vh**, **em**, **rem**, and **px**. These units help create flexible, scalable layouts across different screen sizes.

Experiment 4

```
<html>
```

```
<head>
```

```
<title>Form Validation</title>
```

```
<script>
```

```
function validateForm() {
```

```
    var name = document.getElementById("name").value;
```

```
    if (name == "") {
```

```
        alert("Name must be filled out");
```

```
        return false;
```

```
    }
```

```
    var email = document.getElementById("email").value;
```

```
    var emailPattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
```

```
    if (!email.match(emailPattern)) {
```

```
        alert("Please enter a valid email address");
```

```
        return false;
```

```
    }
```

```
    return true;
```

```

    }

</script>

</head>

<body>

    <form onsubmit="return validateForm()">

        <label for="name">Name:</label>

        <input type="text" id="name" name="name"><br><br>

        <label for="email">Email:</label>

        <input type="email" id="email" name="email"><br><br>

        <input type="submit" value="Submit">

    </form>

</body>

</html>

```

1. The purpose of client-side validation is to check user input for errors before submitting the form to the server, ensuring data is correct, reducing server load, and providing faster feedback to users.
2. A **regular expression** (regex) in JavaScript is a pattern used to match character combinations in strings. It is used for validating, searching, or manipulating text data.
3. Client-side validation improves user experience and reduces server load, while server-side validation ensures data security and integrity. Both are essential for secure and reliable applications.
4. If JavaScript is disabled, interactive features like form validation and dynamic content won't work, and the website may have limited functionality.
5. To handle form errors in JavaScript, validate input fields and show error messages if needed, then prevent form submission if errors exist.

Example : `if (!name) { alert("Name is required"); return false; }`

Experiment 5

2.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note SYSTEM "Note.dtd">
<note>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend!</body>
</note>
```

DTD:

```
<!DOCTYPE note
[
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)><!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
]>
```

1. A **DTD** defines the structure and rules for an XML document, ensuring it is valid.
2. CSS enhances XML presentation by adding styles like colors and layouts, making it visually appealing and easier to read.
3. The **XML Stylesheet Processing Instruction** (<?xml-stylesheet?>) links an XML document to a CSS or XSL stylesheet, specifying how the document should be displayed in the browser.
4. A **well-formed** XML document follows syntax rules. A **valid** XML document is well-formed and conforms to a DTD or XML Schema.
5. To ensure an XML document is valid according to its DTD, include the DTD reference in the document and check for compliance using XML parsers that validate the structure against the DTD.

Experiment 6

```
public class Employee {  
    private int empID;  
    private String name;  
    private double salary;  
    private String designation;  
    private String department;  
    public int getEmpID() { return empID; }  
    public void setEmpID(int empID) { this.empID = empID; }  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
    public double getSalary() { return salary; }  
    public void setSalary(double salary) { this.salary = salary; }  
    public String getDesignation() { return designation; }  
    public void setDesignation(String designation) { this.designation = designation; }  
    public String getDepartment() { return department; }  
    public void setDepartment(String department) { this.department = department; }  
}
```

1. A **JavaBean** is a reusable Java class with private fields, public getters and setters, and a no-argument constructor.
2. The **Serializable** interface allows JavaBeans to be converted into a byte stream for storage or transmission.
3. A JavaBean must follow these conventions:

It must have a **no-argument constructor**.

Its properties should be **private**.

It should provide **public getters and setters** for accessing properties.

It must be **serializable**.

4. In JavaBeans, **getters** retrieve and **setters** update private variables, ensuring controlled access to an object's properties.

5. Yes, a JavaBean can have additional methods apart from getters and setters, such as business logic or utility methods.

Experiment 7

```
mkdir factorial-cli
```

```
cd factorial-cli
```

```
npm init -y
```

```
touch factorial.js
```

```
function factorial(n)
```

```
{
```

```
  if (n < 0)
```

```
  {
```

```
    return "Factorial is not defined for negative numbers.";
```

```
  }
```

```
  if (n === 0 || n === 1)
```

```
  {
```

```
    return 1;
```

```
  }
```

```
  let result = 1;
```

```
  for (let i = 2; i <= n; i++)
```

```
  {
```

```
    result *= i;
```

```
  } return result;
```

```
}
```

```
const args = process.argv.slice(2);
```

```
const number = parseInt(args[0]);
```

```
if (isNaN(number)) { console.log("Please provide a valid number."); }
```

```
else { const result = factorial(number);
```

```
  console.log(`The factorial of ${number} is ${result}.`);
```

```
}
```

Output

```
node factorial.js 5
```

The factorial of 5 is 120.

```
node factorial.js 0
```

The factorial of 0 is 1

1. `process.argv` in Node.js holds the command-line arguments passed to the script.
2. The `crypto` module in Node.js is used for cryptographic operations like hashing, encryption, and decryption.
3. Add `#!/usr/bin/env node` at the top and run `chmod +x <script.js>` to make a Node.js script executable.
4. Invalid input causes an error message and can exit the program with `process.exit(1)`.

5. Randomness is important in password generation to ensure unpredictability, making it difficult for attackers to guess or crack the password through methods like brute force or pattern recognition.

Experiment 8

// insertUsers.js

```
const { MongoClient } = require('mongodb');

async function run() {

  const client = new MongoClient('mongodb://localhost:27017');

  await client.connect();

  const users = client.db('userDB').collection('users');

  const userData = [

    { name: 'Alice', age: 30, city: 'New York' },

    { name: 'Bob', age: 25, city: 'Los Angeles' },

    { name: 'Charlie', age: 35, city: 'New York' },

    { name: 'David', age: 40, city: 'Chicago' },

    { name: 'Eve', age: 22, city: 'Los Angeles' },

    { name: 'Frank', age: 28, city: 'Chicago' },

    { name: 'Grace', age: 35, city: 'New York' },

  ];

  await users.insertMany(userData);

  console.log('7 users inserted.');
```

```
  await client.close();

}
```

```

run().catch(console.dir);

// aggregateUsers.js

const { MongoClient } = require('mongodb');

async function run() {
  const client = new MongoClient('mongodb://localhost:27017');
  await client.connect();
  const users = client.db('userDB').collection('users');
  const result = await users.aggregate([
    { $group: { _id: '$city', averageAge: { $avg: '$age' }, count: { $sum: 1 } } },
    { $sort: { averageAge: 1 } }
  ]).toArray();

  result.forEach(city => console.log(`City: ${city._id}, Avg Age: ${city.averageAge}, Count: ${city.count}`));

  await client.close();
}

run().catch(console.dir);

```

Output

7 users inserted. (insertUsers.js)

City: Los Angeles, Avg Age: 26.5, Count: 2 (aggregateUsers.js)

City: Chicago, Avg Age: 34, Count: 2

City: New York, Avg Age: 33.33, Count: 3

1. MongoDB's aggregation framework processes data with operations like filtering, grouping, and sorting to produce complex results.
2. The `$match` stage in the aggregation pipeline filters documents based on specified criteria, similar to a `find` query, to pass only the matching documents to the next stage.
3. The `$group` stage groups documents by a specified key and performs aggregation operations (e.g., sum, avg) on grouped data.

4. The `$sort` stage orders documents based on specified fields, either in ascending or descending order.
5. If the MongoDB connection is not properly closed, it can lead to memory leaks, unclosed resources, and potential performance issues, as the connection remains open and consumes system resources.

Experiment 9

// LoginServlet.java

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class LoginServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {

        String users = "user1:pwd1,user2:pwd2,user3:pwd3,user4:pwd4";

        Cookie userCookie = new Cookie("userData", users);

        response.addCookie(userCookie);

        String enteredUser = request.getParameter("username");

        String enteredPwd = request.getParameter("password");

        Cookie[] cookies = request.getCookies();

        if (cookies != null) {

            for (Cookie cookie : cookies) {

                if (cookie.getName().equals("userData")) {

                    String[] userData = cookie.getValue().split(",");

                    for (String user : userData) {

                        String[] credentials = user.split(":");

                        if (credentials[0].equals(enteredUser) && credentials[1].equals(enteredPwd)) {

                            response.getWriter().write("Authentication successful!");

                            return;
                        }
                    }
                }
            }
        }
    }
}
```

```

        }
    }
}

}

}

response.getWriter().write("Authentication failed!");
}
}

```

1. A cookie is small data stored in a user's browser to remember information between requests.
2. A servlet creates cookies to store user data and retrieves them for authentication in subsequent requests.
3. Storing sensitive data in cookies risks session hijacking, XSS, CSRF, and cookie theft if not properly secured.
4. Set the `secure` flag on the cookie to ensure it is sent only over HTTPS.
5. The `setMaxAge` method sets the expiration time for a cookie, determining how long the cookie is valid before it is deleted.

Experiment 10

// RegistrationServlet.java

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
import java.io.*;
```

```
import java.sql.*;
```

```
public class RegistrationServlet extends HttpServlet {
```

```
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
        ServletException, IOException {
```

```
        String name = request.getParameter("name");
```

```
        String password = request.getParameter("password");
```

```

String email = request.getParameter("email");

String phone = request.getParameter("phone");

try {

    Class.forName("com.mysql.cj.jdbc.Driver");

    Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/yourdb", "username",
"password");

    String query = "INSERT INTO users (name, password, email, phone) VALUES (?, ?,
?, ?)";

    PreparedStatement pst = con.prepareStatement(query);

    pst.setString(1, name);

    pst.setString(2, password);

    pst.setString(3, email);

    pst.setString(4, phone);

    pst.executeUpdate();

    response.sendRedirect("success.jsp");

} catch (Exception e) {

    e.printStackTrace();

}

}

}

```

1. A servlet handles client requests, processes them, and generates dynamic responses in a web application.
2. `doGet` handles HTTP GET requests (retrieving data), while `doPost` handles HTTP POST requests (sending data).
3. JSP is for presentation (HTML), while Servlets handle request processing and logic.
4. JDBC (Java Database Connectivity) allows Java applications to connect to and interact with databases by executing SQL queries and retrieving results.
5. Hashing passwords ensures they are stored securely, making it difficult for attackers to recover the original password if the database is compromised.

Experiment 11

```
<%@ page import="java.sql.*" %>

<html>

<%

    String action = request.getParameter("action");

    String username = request.getParameter("username");

    String password = request.getParameter("password");

    try {

        Class.forName("com.mysql.cj.jdbc.Driver");

        Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/yourdb",
"username", "password");

        if ("register".equals(action)) {

            PreparedStatement pst = con.prepareStatement("INSERT INTO users (username, password)
VALUES (?, ?)");

            pst.setString(1, username);

            pst.setString(2, password);

            pst.executeUpdate();

            response.sendRedirect("index.jsp");

        } else if ("login".equals(action)) {

            PreparedStatement pst = con.prepareStatement("SELECT * FROM users WHERE username=?
AND password=?");

            pst.setString(1, username);

            pst.setString(2, password);

            ResultSet rs = pst.executeQuery();

            out.println(rs.next() ? "Login Success" : "Invalid Credentials");

        }

    } catch (Exception e) { e.printStackTrace(); }

%>

<form action="index.jsp" method="post">
```

```
<input type="hidden" name="action" value="register"/>

Username: <input type="text" name="username"/><br>

Password: <input type="password" name="password"/><br>

<input type="submit" value="Register"/>

</form>
```

```
<form action="index.jsp" method="post">

  <input type="hidden" name="action" value="login"/>

  Username: <input type="text" name="username"/><br>

  Password: <input type="password" name="password"/><br>

  <input type="submit" value="Login"/>

</form>

</html>
```

1. JSP dynamically generates HTML content for web applications, separating presentation logic from business logic.
2. JDBC provides an API that allows Java applications to connect to, execute queries, and interact with databases.
3. The GET method retrieves data from the server, while the POST method sends data to the server for processing.
4. Use hashing (e.g., bcrypt) and salt passwords before storing them, and ensure secure connections (HTTPS) for data transmission.
5. Prepared statements prevent SQL injection by using parameterized queries, ensuring user input is treated as data, not executable code.

Experiment 12

```
<!-- cart.jsp -->

<%@ page import="java.util.*, javax.servlet.http.*, javax.servlet.*" %>

<html>

<%

    HttpSession session = request.getSession(true);

    List<String> cart = (List<String>) session.getAttribute("cart");

    if (cart == null) {

        cart = new ArrayList<String>();

        session.setAttribute("cart", cart);

    }

    String item = request.getParameter("item");

    if (item != null) {

        cart.add(item);

    }

%>

<form method="post" action="cart.jsp">

    Item: <input type="text" name="item"/>

    <input type="submit" value="Add to Cart"/>

</form>

<h2>Your Cart:</h2>

<ul>

<%

    for (String i : cart) {

        out.println("<li>" + i + "</li>");

    }

%>
```

```
</ul>

</html>

<!-- index.jsp -->

<a href="cart.jsp?item=Item1">Add Item 1</a><br>

<a href="cart.jsp?item=Item2">Add Item 2</a><br>

<a href="cart.jsp?item=Item3">Add Item 3</a><br>
```

1. Local storage persists data across sessions, while session storage only lasts for the duration of the page session.
2. The `fetch` API in JavaScript allows making network requests to retrieve or send data to a server, returning a Promise that resolves to the `Response` object.
3. APIs in web development provide advantages such as enabling data sharing between systems, improving scalability, allowing third-party integrations, and promoting modularity and reusability.
4. You handle API errors in JavaScript using `.catch()` for Promises or `try...catch` with `async/await` to catch and manage exceptions.
5. Session tracking in a shopping cart system keeps track of a user's cart items across multiple pages or visits.